Lean-Backed NS Interface Note

April 1, 2026

Abstract

This note records only the Navier–Stokes material currently backed by Lean in the MeTTapedia repository. It intentionally omits manuscript-specific claims about Ben Goertzel's December 11, 2025 SG-Cole-Hopf draft unless they have been encoded and checked in Lean.

The present Lean status is interface-level rather than proof-complete. The formalized material isolates reusable shells for cutoff continuity, topological Cole–Hopf transport, approximation interfaces, finite identity-energy algebra, abstract vorticity bounds, a kernel-semigroup wrapper, and two small concrete models showing both positive approximation instantiations and wrong topology obstructions, including one exact same-state-space split where a summable geometric observable is continuous and a stronger tail detector is not. It also now contains an abstract good observable class, an abstract tail-spike no-go theorem, and a first concrete soft-vs-hard chart fork on that same shared mode space, together with more manuscript-shaped Sobolev-style and energy-style soft-vs-hard forks on that same space, plus a log-energy fork showing that a smoother radial profile still lands on the same good/bad split, and finally a generic energy-profile theorem packaging that split for any continuous radial profile dominated by the identity on nonnegative inputs. It also now contains a theorem-statement-back interface from a local Fefferman-style global-regularity target to the exact uniform vorticity-control tendril currently produced by the grassroots packages, together with a constructive top-down bridge interface decomposing the missing lift into candidate fields, regularity, dynamics, energy, and vorticity-compatibility layers. The newest pass also specializes that compatibility frontier back down to the concrete windowed Cole–Hopf heat route: a genuine smooth window still survives the present approximation and heat-envelope shells, and the resulting scaled and affine seed-blend descendants already satisfy bridge-level rigidity theorems for original same-route closure. It does *not* yet formalize the manuscript's specific chart, local hypotheses $(H1)$, $(H2)$, or the BKM-closing analytic bridge.

1 Scope

The Navier–Stokes Lean work currently lives in:

1. `FluidDynamics/NavierStokes/CutoffContinuity.lean`,
2. `FluidDynamics/NavierStokes/WeakTopologyCountermodel.lean`,
3. `FluidDynamics/NavierStokes/ProductTopologyL1Countermodel.lean`,
4. `FluidDynamics/NavierStokes/ColeHopfTopology.lean`,
5. `FluidDynamics/NavierStokes/ApproximationInterface.lean`,
6. `FluidDynamics/NavierStokes/ChartApproximationModel.lean`,
7. `FluidDynamics/NavierStokes/GeometricModeApproximation.lean`,

8. `FluidDynamics/NavierStokes/ModeStateProductTopologyObstruction.lean`,
9. `FluidDynamics/NavierStokes/ModeStateProductTopologyContinuity.lean`,
10. `FluidDynamics/NavierStokes/ModeStateObservableClass.lean`,
11. `FluidDynamics/NavierStokes/ModeStateTailSensitivityObstruction.lean`,
12. `FluidDynamics/NavierStokes/ModeStateChartFork.lean`,
13. `FluidDynamics/NavierStokes/ModeStateSobolevChartFork.lean`,
14. `FluidDynamics/NavierStokes/ModeStateEnergyChartFork.lean`,
15. `FluidDynamics/NavierStokes/ModeStateLogEnergyChartFork.lean`,
16. `FluidDynamics/NavierStokes/ModeStateEnergyProfileFork.lean`,
17. `FluidDynamics/NavierStokes/FeffermanTargetBack.lean`,
18. `FluidDynamics/NavierStokes/FeffermanTopDownBridge.lean`,
19. `FluidDynamics/NavierStokes/FeffermanCompatibilityFrontier.lean`,
20. `FluidDynamics/NavierStokes/FeffermanWeakSelfModel.lean`,
21. `FluidDynamics/NavierStokes/FeffermanSeededSelfModel.lean`,
22. `FluidDynamics/NavierStokes/FeffermanBoundedSeededSelfModel.lean`,
23. `FluidDynamics/NavierStokes/FeffermanPressureSeededSelfModel.lean`,
24. `FluidDynamics/NavierStokes/FeffermanScaledPressureSeededFrontier.lean`,
25. `FluidDynamics/NavierStokes/FeffermanSeedBlendPressureFrontier.lean`,
26. `FluidDynamics/NavierStokes/FeffermanSaturatedPressureFrontier.lean`,
27. `FluidDynamics/NavierStokes/FeffermanFrozenGatePressureFrontier.lean`,
28. `FluidDynamics/NavierStokes/FeffermanSeedLiveProfileFrontier.lean`,
29. `FluidDynamics/NavierStokes/FeffermanSeedLiveOperatorFrontier.lean`,
30. `FluidDynamics/NavierStokes/WindowedCutoffApproximation.lean`,
31. `FluidDynamics/NavierStokes/WindowedColeHopfHeatPackage.lean`,
32. `FluidDynamics/NavierStokes/WindowedColeHopfHeatFoundationFrontier.lean`,
33. `FluidDynamics/NavierStokes/WindowedColeHopfHeatSeededModel.lean`,
34. `FluidDynamics/NavierStokes/WindowedColeHopfHeatScaledSeededFrontier.lean`,
35. `FluidDynamics/NavierStokes/WindowedColeHopfHeatSelfCompatibilityObstruction.lean`,
36. `FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedBlendFrontier.lean`,

37. `FluidDynamics/NavierStokes/WindowedColeHopfHeatSeedBlendObstruction.lean`,
38. `FluidDynamics/NavierStokes/IdentityEnergy.lean`,
39. `FluidDynamics/NavierStokes/ColeHopfInterface.lean`,
40. `FluidDynamics/NavierStokes/KernelSemigroupInterface.lean`.

These modules form the live NS interface shell. The currently verified slice is recorded explicitly in Section 4. Together they provide a formal shell for what a future SG-Cole-Hopf proof attempt would need to instantiate.

2 Lean-Backed Results

2.1 Cutoff Continuity Shell

In `CutoffContinuity.lean`, the generic cutoff-shaped potential

$$\mathrm{cutoffPotential}(x) = \mathrm{cutoff}(\mathrm{radius}(x)) \cdot \mathrm{statistic}(x)$$

is formalized together with:

1. continuity of the cutoff potential when the scalar pieces are continuous;
2. a sequential obstruction theorem: if the cutoff factor and statistic converge to the wrong product, then the cutoff potential cannot converge to its claimed value;
3. the corresponding pointwise non-continuity criterion.

So Lean already captures the generic shape of a chart-cutoff continuity argument and its failure mode.

2.2 Concrete Weak-Topology Countermodel

In `WeakTopologyCountermodel.lean`, the abstract cutoff/Cole–Hopf shell is instantiated on the concrete state space $\mathbb{R} \times \mathbb{R}$ equipped with the topology induced by the first coordinate. Lean proves:

1. a sequence can converge in the ambient topology while changing a hidden coordinate;
2. a cutoff potential depending on that hidden coordinate is not continuous;
3. the associated Cole–Hopf transform is likewise not continuous.

This is a formal countermodel for the generic wrong-topology failure mode: ambient convergence can be too weak for the cutoff datum it is supposed to control.

2.3 Infinite-Mode Product-Topology Countermodel

In `ProductTopologyL1Countermodel.lean`, Lean moves the same mismatch to the raw infinite-mode sequence space $\mathbb{N} \to \mathbb{R}$ with the product topology. It proves:

1. finite-mode truncations converge coordinatewise in the product topology;
2. a tail spike moving to higher and higher coordinates converges to the zero sequence in that topology;

3. the stronger tail-sensitive observable $\sum_n |x_n|$ stays equal to 1 on that spike sequence and is therefore not continuous at zero;
4. the associated Cole–Hopf datum is likewise not continuous.

This is the first Lean-backed infinite-mode obstruction in the NS lane that directly separates weak coordinatewise convergence from a stronger tail norm.

2.4 Topological Cole–Hopf Shell

In `ColeHopfTopology.lean`, Lean formalizes:

1. continuity of the scalar map $y \mapsto \exp(-y/(2\nu))$;
2. continuity of the induced field-level Cole–Hopf transform;
3. injectivity when $\nu \neq 0$;
4. transfer of potential-limit mismatch into Cole–Hopf-limit mismatch;
5. the same obstruction specialized to cutoff-shaped potentials.

This means the topological $W \mapsto \phi$ step is already packaged in Lean, provided one supplies continuity or convergence of the underlying potential.

2.5 Approximation Interface

In `ApproximationInterface.lean`, Lean packages an abstract positive repair obligation:

> if an approximation scheme makes the chart radius observable and the chart statistic converge, then the cutoff potential and the Cole–Hopf datum also converge.

This is useful because it isolates exactly what an approximation/truncation theorem would have to prove in order for the cutoff/Cole–Hopf layer to behave well.

2.6 Concrete Chart/Truncation Model

In `ChartApproximationModel.lean`, the abstract approximation interface is instantiated on the concrete chart state space $\mathbb{R} \times \mathbb{R}$ with:

1. a hidden-coordinate radius observable;
2. a simple mixed chart statistic;
3. a truncation map that drops the hidden coordinate only at stage 0 and is exact thereafter.

Lean then proves:

1. convergence of the radius observable under truncation;
2. convergence of the chart statistic under truncation;
3. the resulting convergence of the cutoff potential;
4. the resulting convergence of the induced Cole–Hopf datum.

So the approximation interface is now backed by a concrete positive model, not just an abstract shell.

2.7 Geometric Finite-Mode Approximation Model

In `GeometricModeApproximation.lean`, the same approximation interface is instantiated on bounded infinite coefficient sequences. Lean formalizes:

1. a geometrically weighted mode radius;
2. a chart statistic built from the zeroth visible mode plus that radius;
3. finite-mode truncation keeping only the first N coefficients;
4. summability of the weighted mode series under the uniform coefficient bound;
5. convergence of the weighted radius under finite-mode truncation;
6. convergence of the derived chart statistic, the cutoff potential, and the induced Cole–Hopf datum.

This is the first genuinely infinite-mode positive instantiation in the NS Lean lane: the approximation is not just "wrong at stage 0, exact thereafter," but a real truncation-to-limit model on an infinite sequence state space.

2.8 Same-State-Space Product-Topology Obstruction

In `ModeStateProductTopologyObstruction.lean`, Lean returns to the exact `ModeState` and `truncateModes` objects from `GeometricModeApproximation.lean`, but equips that same state space with the coordinatewise product topology induced by the coefficient map. It proves:

1. the existing finite-mode truncations `truncateModes` do converge to the target state in this product topology;
2. a spike sequence still converges to the zero state in that same topology;
3. a stronger tail-presence observable, which records whether any nonzero coefficient exists, is not continuous there;
4. the associated Cole–Hopf datum is likewise not continuous.

So Lean now contains a positive and a negative result on one common infinite-mode object: the geometric weighted radius behaves well under truncation, while a stronger tail-sensitive observable on the same state space does not.

2.9 Same-State-Space Product-Topology Continuity

In `ModeStateProductTopologyContinuity.lean`, Lean proves the matching positive theorem on the exact same `ModeState` equipped with the exact same coordinatewise product topology:

1. each coordinate observable $x \mapsto x_n$ is continuous;
2. each weighted mode term $x \mapsto |x_n|2^{-n}$ is continuous;
3. the summable geometric radius $\sum_n |x_n|2^{-n}$ is continuous;
4. the derived chart statistic, cutoff potential, and Cole–Hopf datum are therefore continuous as well.

So the current NS Lean lane now cleanly separates two observables on one common infinite-mode state space: a summably dominated geometric radius that behaves well in the product topology, and a stronger tail detector that does not.

2.10 Abstract Good Observable Class

In `ModeStateObservableClass.lean`, Lean packages the good side on the shared `ModeState`: nonnegative summable mode weights. For that whole class, Lean proves:

1. continuity of the weighted radius observable;
2. continuity of the derived statistic, cutoff potential, and Cole–Hopf datum;
3. convergence of all of these along the existing truncations `truncateModes`;
4. recovery of the earlier geometric radius model as a special case.

So the good side is no longer just one geometric example. It is now an abstract summable observable class on the shared mode space.

2.11 Abstract Tail-Spike No-Go Theorem

In `ModeStateTailSensitivityObstruction.lean`, Lean packages the bad side abstractly too. If an observable is constant on every remote single-spike state `tailSpikeMode`(N) but takes a different value at `modeZero`, Lean proves:

1. the raw observable is not continuous at `modeZero`;
2. the same obstruction lifts to cutoff potentials;
3. the same obstruction lifts further to Cole–Hopf data.

So the bad side is no longer tied only to the one hand-built `tailPresenceRadius`. It is now a reusable theorem pattern.

2.12 First Concrete Chart Fork

In `ModeStateChartFork.lean`, Lean forces the first explicit chart-style fork on the shared mode space:

1. the soft support observable $t \mapsto |t|/(1+|t|)$, combined with summable mode weights, lies on the good side and yields a continuous truncation-friendly chart radius, statistic, cutoff potential, and Cole–Hopf datum;
2. the hard support observable, which only asks whether any coefficient is nonzero, lies on the bad side and fails continuity at `modeZero`, together with its cutoff and Cole–Hopf lifts.

This is the first genuine theorem-level NS fork in Lean:

> soft support belongs to the good summable side, hard support belongs to the bad tail-sensitive side.

2.13 Sobolev-Style Chart Fork

In `ModeStateSobolevChartFork.lean`, Lean pushes that fork one step closer to manuscript shape by amplifying each mode by the polynomial factor $((n+1)^m)$ before applying the chart cutoff:

1. the soft Sobolev observable, built from the bounded scalar $t \mapsto |t|/(1+|t|)$ together with a concrete inverse-square envelope, is continuous and truncation-friendly;

2. the corresponding hard Sobolev observable, which only asks whether any amplified mode is nonzero, is proved equal to the earlier hard support detector and is therefore not continuous at `modeZero`, together with its cutoff and Cole–Hopf lifts.

   So the current Lean fork is now sharper:

   > even after Sobolev-style high-frequency amplification, the soft bounded version lands on the good summable side, while the hard support version still lands on the bad side.

2.14 Energy-Style Chart Fork

In `ModeStateEnergyChartFork.lean`, Lean pushes the same shared-state fork one step further toward an energy-style observable:

1. the soft energy observable squares the Sobolev-amplified modes and then applies a compensating inverse-power envelope; Lean proves that this radius is continuous and truncation-friendly, and so are its cutoff and Cole–Hopf lifts;

2. the hard energy observable asks whether any amplified squared mode crosses the threshold 1; Lean proves that this detector is stable on the remote spike family, disagrees at `modeZero`, and is therefore not continuous, again together with its cutoff and Cole–Hopf lifts.

   So the current shared-mode-space fork is now sharper:

   > if the chart observable keeps enough inverse-power compensation to remain summable after Sobolev amplification and squaring, Lean puts it on the good approximation-friendly side; if it turns into a hard energy threshold on those amplified modes, Lean puts it on the bad tail-sensitive side.

2.15 Log-Energy Chart Fork

In `ModeStateLogEnergyChartFork.lean`, Lean tests a smoother radial profile on the same shared mode space by replacing raw squared Sobolev energy by $\log(1+\text{energy})$:

1. the soft log-energy observable is continuous and truncation-friendly; the key formal domination is $\log(1+u) \le u$, which lets Lean compare the log-energy terms to the already-controlled soft energy terms;

2. the hard-side analogue asking whether $\log(1+\text{energy}) > 0$ still lands on the bad side: it is stable on the remote spike family, disagrees at `modeZero`, and therefore its cutoff and Cole–Hopf lifts are also not continuous.

   So the shared-mode-space fork is now sharper again:

   > even replacing raw energy by the smoother profile $\log(1+\text{energy})$ does not escape the same dichotomy: the summably compensated soft version stays on the good side, while the hard detection version stays on the bad side.

2.16 Generic Energy-Profile Fork

In `ModeStateEnergyProfileFork.lean`, Lean abstracts the preceding examples into one theorem pattern. An `EnergyProfile` is a globally continuous radial profile $\rho : \mathbb{R} \to \mathbb{R}$ satisfying, on nonnegative inputs:

1. $0 \leq \rho(u)$,
2. $\rho(u) \leq u$,
3. $\rho(0) = 0$,
4. $u > 0 \Rightarrow \rho(u) > 0$.

For every such profile, Lean proves:

1. the summably compensated soft radius is continuous and truncation-friendly;
2. the induced hard detector agrees with the raw hard Sobolev support detector;
3. that hard detector is tail-spike stable and therefore not continuous at `modeZero`, together with its cutoff and Cole–Hopf lifts.

The file also includes canonical instances:

1. the identity profile, recovering the raw soft energy radius;
2. the clipped log profile $u \mapsto \log(1 + \max(u, 0))$, recovering the soft and hard log-energy constructions on the nonnegative energy inputs that actually occur;
3. a bounded rational saturation profile $u \mapsto \max(u, 0)/(1 + \max(u, 0))$, which is closer to a chart-cutoff saturation shape and whose hard detector Lean proves collapses back to the hard Sobolev support detector;
4. a smooth exponential saturation profile $u \mapsto 1 - \exp(-\max(u, 0))$, again with the same hard-detector collapse;
5. a smooth-transition profile $u \mapsto u \cdot \mathrm{smoothTransition}(u)$, using mathlib's `Real.smoothTransition`, again with the same hard-detector collapse;
6. more strongly, a positive-parameter family $u \mapsto u \cdot \mathrm{smoothTransition}(cu)$ for $c > 0$, all with the same hard-detector collapse;
7. stronger still, an affine two-parameter family $u \mapsto u \cdot \mathrm{smoothTransition}(cu + d)$ for $c > 0$, $d \geq 0$, all with the same hard-detector collapse.

The new boundary result is that a genuinely windowed difference-of-transitions profile already leaves the shell. Lean defines

$$u \mapsto u \cdot \big(\mathrm{smoothTransition}(cu + 1) - \mathrm{smoothTransition}(cu)\big)$$

and proves that for every $c > 0$ this profile cannot be the `profile`-field of any `EnergyProfile`: it vanishes at the positive input $u = c^{-1}$, so it cannot satisfy the required strict positivity on all positive energies.

So the current shared-mode-space NS fork is now theorem-level rather than example-level:

> every continuous radial profile that stays nonnegative, vanishes only at zero, and is dominated by the identity on nonnegative inputs lands on the same good/bad fork.

2.17 Finite Identity-Energy Algebra

In `IdentityEnergy.lean`, Lean formalizes a finite carré-du-champ-style energy

$$\gamma(D) = \sum_i D_i^2$$

and proves:

1. basic positivity and scaling laws for γ;
2. the log-gradient comparison used in the identity-only Cole–Hopf calculation;
3. the finite Cauchy–Schwarz frame-evaluation bound;
4. the resulting pointwise vorticity-style estimate after push-down through a curl frame.

   So the finite algebraic spine of the identity-only bound is already Lean-backed.

2.18 Abstract Cole–Hopf/Vorticity Interface

In `ColeHopfInterface.lean`, the finite identity-energy algebra is packaged into an abstract structure carrying:

1. uniform positivity of $\Phi(t)$;
2. a uniform identity-energy bound on its directional derivatives;
3. a uniform frame-curl bound.

   From those hypotheses, Lean derives:

1. a bound on the log-potential coefficients $W(t) = -2\nu \log \Phi(t)$;
2. a pointwise abstract vorticity bound;
3. the uniform-in-time version of the same estimate.

2.19 Kernel-Semigroup Wrapper

In `KernelSemigroupInterface.lean`, the same identity-only Cole–Hopf/vorticity algebra is wrapped around a local Markov-kernel semigroup. This does not add new analytic content. It says that if a future NS evolution is encoded as a local kernel semigroup, then the existing identity-energy bridge can be reused unchanged.

2.20 Fefferman-Style Target Backward Interface

In `FeffermanTargetBack.lean`, Lean now works backward from a local mirror of the Clay/Fefferman output clause. The file first defines a `FeffermanPredicateKit`, so the target is parameterized by abstract output-side predicates but still requires actual proofs of them. It then does three things:

1. it defines `UniformVorticityTendril`, the exact smallest theorem-statement-back fragment currently reached by the grassroots lane: a globally defined vorticity field with one uniform envelope;

2. it proves that the existing local shells `ColeHopfIdentityData`, `FiniteModeColeHopfData`, `ColeHopfKernelSemigroupData`, `FiniteModeColeHopfKernelData`, `ManuscriptTruncationPackage`, and `SharedApproximationPackage` all land in that same tendril;

3. it defines `FeffermanLiftObligations`, the exact additional lift still missing if one wants to climb from that tendril to a local Fefferman-style global-regularity output witness: velocity, pressure, proofs of the chosen smoothness / dynamics / energy predicates, and a proof of the chosen vorticity-to-velocity compatibility predicate.

So the target-back picture is now explicit in Lean:

> current grassroots NS packages already produce a uniform vorticity-control tendril, but a separate lift is still required before one has even a local mirror of the Fefferman-style global-regularity output clause.

2.21 Top-Down Fefferman Bridge

In `FeffermanTopDownBridge.lean`, Lean turns that missing lift into a positive program rather than a vague gap. Relative to a chosen predicate kit and a chosen vorticity-compatibility predicate, the file defines:

1. `VelocityPressureCandidate`, the candidate output fields;

2. `RegularityCertificate`, `DynamicsCertificate`, and `EnergyCertificate`, separating smoothness, PDE/incompressibility / initial-condition, and bounded-energy obligations;

3. `VorticityCompatibility`, isolating the extra theorem that ties the already-available vorticity tendril back to the chosen velocity field;

4. `TopDownFeffermanBridge`, bundling exactly those layers.

Lean then proves that any such bridge assembles:

1. the explicit `FeffermanLiftObligations` from the target-back file;

2. the local Fefferman-style global-regularity clause;

3. preservation of the original uniform vorticity tendril.

Finally, the file proves package-level realization theorems for the current NS lane: if a top-down bridge is supplied on the tendril induced by `ColeHopfIdentityData`, `FiniteModeColeHopfData`, `ColeHopfKernelSemigroupData`, `FiniteModeColeHopfKernelData`, `ManuscriptTruncationPackage`, or `SharedApproximationPackage`, then the local Fefferman-style clause follows immediately.

So the constructive NS picture is now also explicit in Lean:

> the current grassroots route already reaches a uniform vorticity tendril, and the remaining top-down work is decomposed into theorem-sized lift layers rather than one monolithic miracle step.

2.22 Compatibility Frontier

In `FeffermanCompatibilityFrontier.lean`, Lean peels that top-down bridge one step further. It defines `AlmostFeffermanBridge`, which contains:

1. a candidate velocity/pressure pair;
2. the corresponding regularity certificate;
3. the corresponding dynamics certificate;
4. the corresponding energy certificate;

but deliberately omits vorticity-to-velocity compatibility.

Lean then proves that once a compatibility proof is supplied, the full `TopDownFeffermanBridge` and hence the local Fefferman-style clause follow immediately. It also adds package-level reduction theorems showing that `ColeHopfIdentityData`, `FiniteModeColeHopfData`, `ColeHopfKernelSemigroupData`, `FiniteModeColeHopfKernelData`, `ManuscriptTruncationPackage`, and `SharedApproximationPackage` all reduce to that same compatibility frontier.

So the live constructive NS bottleneck is now even sharper in Lean:

> once candidate fields and their certificates are fixed, the remaining theorem is exactly the compatibility of the current vorticity tendril with the chosen velocity field.

2.23 Weak Self-Model Realization

In `FeffermanWeakSelfModel.lean`, Lean now proves the top-down stack is not vacuous. The file defines a weak but nontrivial predicate kit where:

1. smoothness, momentum, incompressibility, and initial-condition predicates are left trivial;
2. the only nontrivial output-side predicate is a uniform pointwise bound on the velocity field;
3. compatibility means the chosen velocity field is literally the current vorticity tendril.

With that choice, Lean constructs a canonical self-model:

1. velocity is the tendril vorticity itself;
2. pressure is identically zero;
3. the tendril envelope supplies the boundedness certificate;
4. compatibility is tautological.

It then proves:

1. every `UniformVorticityTendril` realizes the weak local Fefferman-style clause;
2. the same realization can be seen directly through the compatibility frontier;
3. all current NS packages therefore already realize that weak clause.

So the constructive picture is now stronger:

> the top-down NS bridge is not merely a decomposition of obligations; for a weak but genuine target, the current grassroots tendril already closes the whole stack in Lean.

2.24 Seeded and Bounded Seeded Strengthenings

In `FeffermanSeededSelfModel.lean`, Lean strengthens the weak self-model in two concrete ways:

1. the pressure predicate is no longer trivial: it must be identically zero;
2. the initial-condition predicate is no longer trivial: velocity at the distinguished base time 1 must match the seed slice extracted from the tendril itself.

The file proves that every current NS package still realizes that stronger seeded clause by the same self candidate and the same compatibility proof.

In `FeffermanBoundedSeededSelfModel.lean`, Lean strengthens the target once more by making the velocity-side regularity predicate nontrivial too: it must satisfy the same uniform pointwise bound already carried by the tendril envelope. The same self candidate still closes the clause.

In `FeffermanPressureSeededSelfModel.lean`, Lean strengthens the target again by making one more dynamics field nontrivial: the momentum slot now also requires the pressure field to vanish. Since the canonical self candidate still has identically zero pressure, the same self candidate and the same compatibility proof still close the clause.

So the current top-down picture is sharper:

> the current NS tendril now closes not just a boundedness-only toy target, but a proof-carrying family of self-model targets with zero pressure, explicit initial slice, a nontrivial velocity-bound predicate, and now also a nontrivial momentum-slot pressure condition.

2.25 First Non-Self Compatibility Frontier

In `FeffermanScaledPressureSeededFrontier.lean`, Lean introduces the first genuinely non-self candidate: velocity is a scalar multiple of the current vorticity tendril, while pressure stays identically zero. Lean proves:

1. the same strengthened pressure-seeded target still has full regularity, dynamics, and energy certificates for this rescaled candidate;
2. therefore the same-route version now reaches an explicit `AlmostFeffermanBridge` with a nontrivial compatibility mouth;
3. a nearby descendant route with rescaled compatibility closes immediately;
4. the same-route mouth already closes in special regimes, including the trivial scaling $a = 1$ and the degenerate zero-vorticity case.

So the constructive picture is sharper again:

> Lean now contains not only self-model closures, but also the first explicit non-self candidate whose certificates are complete and whose remaining obstacle is exactly a compatibility theorem.

In `FeffermanSeedBlendPressureFrontier.lean`, Lean pushes this one step further. The new non-self candidate has velocity

$$u(t, x) = a\,\omega(t, x) + b\,\omega(1, x),$$

mixing the live tendril with its seeded slice while keeping pressure zero. Lean proves:

1. the same strengthened pressure-seeded target still has complete regularity, dynamics, and energy certificates for this affine seed-blend candidate;
2. therefore the same-route version again reaches an explicit `AlmostFeffermanBridge` whose live mouth is compatibility;
3. a nearby descendant route with affine seed-blend compatibility closes immediately;
4. the same-route mouth already closes in special cases such as $(a, b) = (1, 0)$ and the zero-vorticity regime.

So the top-down NS tendril now reaches a second, strictly richer non-self frontier in Lean.

In `FeffermanSaturatedPressureFrontier.lean`, Lean adds the first nonlinear non-self candidate:

$$u(t, x) = a \cdot \frac{\omega(t, x)}{1 + |\omega(t, x)|}.$$

The file proves:

1. the saturated profile still satisfies the strengthened pressure-seeded target's regularity, dynamics, and energy slots;
2. the same-route version again reaches an explicit `AlmostFeffermanBridge` with compatibility as the only remaining mouth;
3. a nearby descendant route with saturated compatibility closes immediately;
4. zero vorticity already collapses the same-route mouth.

So the top-down ladder now contains not just linear and affine non-self frontiers, but also a first nonlinear saturated frontier in Lean.

In `FeffermanFrozenGatePressureFrontier.lean`, Lean adds a more manuscript-shaped nonlinear non-self candidate:

$$u(t, x) = a \cdot \frac{|\omega(1, x)|}{1 + |\omega(1, x)|} \cdot \omega(t, x).$$

The live tendril is now modulated by a nonlinear gate computed from the frozen seed slice. The file proves:

1. this frozen-seed-gated candidate still satisfies the strengthened pressure-seeded target's regularity, dynamics, and energy slots;
2. the same-route version again reaches an explicit `AlmostFeffermanBridge` with compatibility as the only remaining mouth;
3. a nearby descendant route with frozen-gate compatibility closes immediately;
4. zero vorticity again collapses the same-route mouth.

So the top-down ladder now contains not just linear, affine, and pointwise nonlinear frontiers, but also a first frozen-seed-gated frontier in Lean.

In `FeffermanSeedLiveProfileFrontier.lean`, Lean then packages these frontiers into one reusable theorem family. The new structure `SeedLiveProfile` covers any pointwise candidate of the form

$$u(t, x) = \Phi(\omega(1, x), \omega(t, x))$$

under the single bound

$$|\Phi(\text{seed}, \text{live})| \leq c(|\text{seed}| + |\text{live}|).$$

The file proves:

1. every such profile automatically carries the strengthened pressure-seeded target through regularity, dynamics, and energy;
2. the same-route version always reaches an explicit `AlmostFeffermanBridge` whose only remaining mouth is compatibility;
3. the descendant route with the exact profile-compatibility predicate closes immediately;
4. the scalar-rescaled, affine seed-blend, nonlinear saturated, and frozen-seed-gated candidates are recovered as concrete instances.

So the NS top-down lane now has not only several explicit non-self frontier examples, but also a reusable seed/live profile shell that subsumes them.

In `FeffermanSeedLiveOperatorFrontier.lean`, Lean pushes this one step further to a genuinely non-pointwise shell. The new structure `SeedLiveOperator` allows the candidate at x to depend on the full seeded and live slices as functions, provided it satisfies a uniform envelope bound in terms of seeded/live slice bounds. The file proves:

1. every such operator automatically carries the strengthened pressure-seeded target through regularity, dynamics, and energy;
2. the same-route version again reaches an explicit `AlmostFeffermanBridge` whose only remaining mouth is compatibility;
3. the descendant route with exact operator compatibility closes immediately;
4. the pointwise `SeedLiveProfile` shell embeds into this operator shell, and a concrete anchored non-pointwise seed/live blend is realized as an instance.

So the top-down NS lane now reaches not only pointwise non-self frontier families, but also a first proof-carrying non-pointwise operator shell.

2.26 Concrete Windowed Descendant Route

The newest specialization step pushes the same top-down frontier back down to the actual concrete windowed heat package. In `WindowedCutoffApproximation.lean`, Lean proves that the genuinely windowed cutoff

$$u \mapsto u \cdot \big(\text{smoothTransition}(cu + 1) - \text{smoothTransition}(cu)\big)$$

still passes through the current approximation shell. So leaving the stricter `EnergyProfile` shell is not yet a failure of truncation convergence or of the induced Cole–Hopf convergence.

In `WindowedColeHopfHeatPackage.lean`, Lean proves the quantitative window bound

$$|\chi_c(u)| \leq c^{-1},$$

which is enough to instantiate the present heat-decayed Cole–Hopf package. So the current concrete route already carries a genuine windowed shared approximation package and a genuine uniform vorticity tendril.

In `WindowedColeHopfHeatFoundationFrontier.lean` and `WindowedColeHopfHeatSeededModel.lean`, Lean then fixes the actual candidate fields on that concrete route:

1. velocity is the package vorticity field;
2. pressure is identically zero;
3. the candidate closes a stronger seeded local target with zero pressure, a distinguished initial slice at time 1, and a uniform pointwise bound.

So the current compatibility frontier is no longer only an abstract theorem schema. It now sits on one explicit concrete windowed route.

2.27 Concrete Windowed Same-Route Rigidity

The next new files show that the concrete route already has theorem-level same-route rigidity.

In `WindowedColeHopfHeatScaledSeededFrontier.lean`, Lean packages the scaled descendant $u(t,x) = a\,\omega(t,x)$ on the concrete windowed route. In `WindowedColeHopfHeatSelfCompatibilityObstruction.le` Lean proves that if this scaled descendant is forced back through the original self-compatibility mouth on a genuinely nonzero vorticity state, then the scaling must satisfy $a = 1$. Equivalently, if $a \neq 1$, then no bridge of that same-route type exists on a nonzero-vorticity state.

In `WindowedColeHopfHeatSeedBlendFrontier.lean`, Lean packages the affine seed-blend descendant

$$u(t,x) = a\,\omega(t,x) + b\,\omega(1,x).$$

In `WindowedColeHopfHeatSeedBlendObstruction.lean`, Lean proves bridge- level rigidity laws for the same-route mouth:

1. a nonzero initial slice forces $a + b = 1$;
2. a witness with zero initial slice but nonzero live vorticity forces $a = 1$;
3. a witness with zero live vorticity but nonzero initial slice forces $b = 0$;
4. violating any of these identities rules out the corresponding same-route bridge on the concrete windowed route.

So the NS crux has now moved one step forward: the issue is no longer merely that abstract non-self frontiers exist. The issue is that on the concrete windowed route, original same-route closure is already provably rigid.

3 Current Boundary

The following are *not* yet formalized in Lean in the present NS lane:

1. the manuscript's specific chart-cutoff datum on SDiff^m;
2. the finite-mode truncation / approximation theorem actually needed for that datum;
3. instantiations of the new top-down bridge layers with real velocity and pressure candidates, real regularity and dynamics certificates, and a real vorticity-to-velocity compatibility theorem beyond the current self, scalar-rescaled, affine seed-blend, nonlinear saturated, frozen-seed-gated, bounded seed/live operator toy candidates, and the current concrete windowed descendants;

4. strengthening the current seeded / bounded-seeded self models toward something recognizably closer to the Clay/Fefferman predicates than pointwise bounds, zero pressure, and a distinguished initial slice;
5. the local hypotheses $(H1)$, $(H2)$;
6. the manuscript's claimed identity-only propagation lemma;
7. the BKM-closing bridge for the full infinite-dimensional argument.

So the honest current status is:

> Lean has formalized the generic interface shell that a proof attempt would need, but not the manuscript-specific analytic instantiations.

4 Verification

The currently relevant verified NS slice was checked with a grouped build over the `NavierStokes` directory, excluding only two unrelated in-progress local frontier files that are currently broken. Concretely:

```
mods=$(find Mettapedia/FluidDynamics/NavierStokes -maxdepth 1 -name '*.lean' \
  ! -name 'FeffermanSeededSelfModel.lean' \
  ! -name 'FeffermanSeedLiveOperatorFrontier.lean' \
  | sort | sed 's|/|.|g; s|\.lean$||')
lake build $mods
```

5 Next Lean Target

The natural next formal step is no longer more prose. It is to keep pushing top-down from the target:

1. strengthen the proof-carrying target again by making at least one more regularity or dynamics predicate genuinely nontrivial while keeping the clause closable by current packages;
2. push the concrete windowed route beyond same-route rigidity and toward a real velocity-generation or pressure-generation theorem on that fixed candidate;
3. reconnect whichever concrete descendant survives that test to the mode-space chart / approximation shells.